

CharityMap

Application de cartographie pour la répartition de l'aide caritative

Cadre :	Projet PGL (Delaunay / Voronoï) — ING1-GI3-J
Année :	2025 / 2026
Dépôt distant :	https://github.com/Fufulino/Genie-Logiciel.git
Date de remise :	Juin 2026

Présenté par l'équipe GI3J :

Amir, Nassim, Adou, Nicolas, Enzo

1. Présentation du projet

Le projet CharityMap consiste à modéliser et visualiser sur une carte plane interactive en deux dimensions la répartition territoriale de l'aide caritative. Face à un ensemble de centres d'aide existants et de personnes nécessitant une assistance spécifique (nutrition, soins médicaux ou vêtements), l'application détermine et trace instantanément les zones de couverture optimales.

Pour répondre à ce besoin d'équité territoriale et d'efficacité logistique, l'application s'appuie sur deux concepts mathématiques géométriques forts et duaux :

- **Triangulation de Delaunay** : Relie les centres de secours sous forme de maillage triangulaire sans qu'aucun autre centre ne se trouve à l'intérieur du cercle circonscrit de chaque triangle.
- **Diagramme de Voronoï** : Représente la partition géométrique de l'espace en cellules. Chaque point de la cellule d'un centre est plus proche de celui-ci que de tout autre centre compatible.

Notre adaptation du sujet est ainsi résolument orientée vers une problématique sociale et éthique : fournir un outil d'aide à la décision pour permettre aux organisations humanitaires de détecter les zones sous-équipées ou surchargées et d'optimiser l'accès aux ressources d'aide.

2. Organisation du travail

Pour assurer une collaboration fluide et modulaire, le projet a été structuré selon les packages métiers de la programmation orientée objet. Cette répartition a évité les conflits d'édition et a permis à chacun de prendre en main un lot technique précis.

Membre de l'équipe	Responsabilité principale	Exemples de classes développées
Nassim	Interface graphique et géométrie de base	CharityMapApp, MapCanvas, Point, Triangle, GeometryUtils
Amir	Moteur de calcul géométrique et rendu graphique	DelaunayTriangulator, VoronoiBuilder, MapPainter
Adou	Entités métiers et formulaires d'interface	DistributionCenter, Beneficiary, Association, AidType, ControlPanel
Nicolas	Services d'importation, persistance et statistiques	MapIO, RandomGenerator, DetailsPanel, VoronoiCell
Enzo	Moteur logique global et application console	CharityMap, CommandLineApp

3. Fonctionnalités réalisées

3.1 Gestion interactive des entités

L'utilisateur dispose d'une liberté totale de saisie sur la carte. Il peut positionner graphiquement ou manuellement des centres de distribution appartenant à diverses associations et gérant une aide exclusive. Les bénéficiaires sont affectés en temps réel au centre adéquat le plus proche. Le déplacement d'un point par glisser-déposer met à jour dynamiquement l'arborescence géométrique de couverture sans latence.

3.2 Traitement, Import & Génération

Pour charger des configurations géographiques complexes, un module d'importation de fichiers CSV tolère les lignes malformées et ignore les commentaires. L'utilisateur peut aussi déclencher l'apparition de bénéficiaires simulés de façon aléatoire sur la boîte englobante de la ville de Cergy, servant d'outil d'évaluation de charge (stress-test visuel).

3.3 Dualité Delaunay / Voronoï par type d'aide

L'algorithme de Bowyer-Watson calcule le maillage triangulaire de Delaunay sur l'ensemble des centres d'un type d'aide donné. Par dualité géométrique, le programme relie les centres des cercles circonscrits pour ériger les frontières polygonales des cellules de Voronoï. Le calcul est strictement cloisonné par type de besoin (Nourriture, Soins, Vêtements) pour éviter d'associer un besoin médical à un centre alimentaire.

3.4 Double interface applicative

L'application est exécutable sous deux formes :

- **Interface Graphique (GUI) JavaFX** : Carte interactive avec gestion des couches d'affichage (grille, Delaunay, Voronoï, liens), panneau d'action de gauche (CRUD) et panneau d'inspection de droite (statistiques de cellule, aire, distances de trajet).
- **Interface Console (CLI)** : Boucle interactive textuelle affichant des menus structurés de contrôle et émettant des tableaux de statistiques et des rapports de charge directement sur le terminal de commande.

4. Architecture de l'application

L'architecture de l'application repose sur un découpage modulaire strict respectant l'organisation en packages de l'ingénierie logicielle.

Package	Rôle d'encapsulation	Classes clés contenues
<code>com.charitymap.model</code>	Entités métiers et données structurelles	<code>CharityMap</code> , <code>DistributionCenter</code> , <code>Beneficiary</code> , <code>Association</code> , <code>AidType</code> , <code>Point</code>
<code>com.charitymap.geometry</code>	Moteurs algorithmiques et géométriques purs	<code>DelaunayTriangulator</code> , <code>VoronoiBuilder</code> , <code>Triangle</code> , <code>VoronoiCell</code> ,

		GeometryUtils
com.charitymap.service	Modules de traitement de données et persistance	CsvImporter, MapIO, RandomGenerator
com.charitymap.gui	Composants graphiques, Canvas de rendu et style CSS	CharityMapApp, ControlPanel, MapCanvas, DetailsPanel, MapPainter, GeoProjection
com.charitymap.cli	Boucle de contrôle console interactive	CommandLineApp

Diagramme de classes simplifié

Ce diagramme de classes illustre la dualité structurelle entre les entités métier (packages model et service) et le moteur de calcul géométrique (package geometry), tout en assurant l'interaction avec les interfaces GUI et CLI.

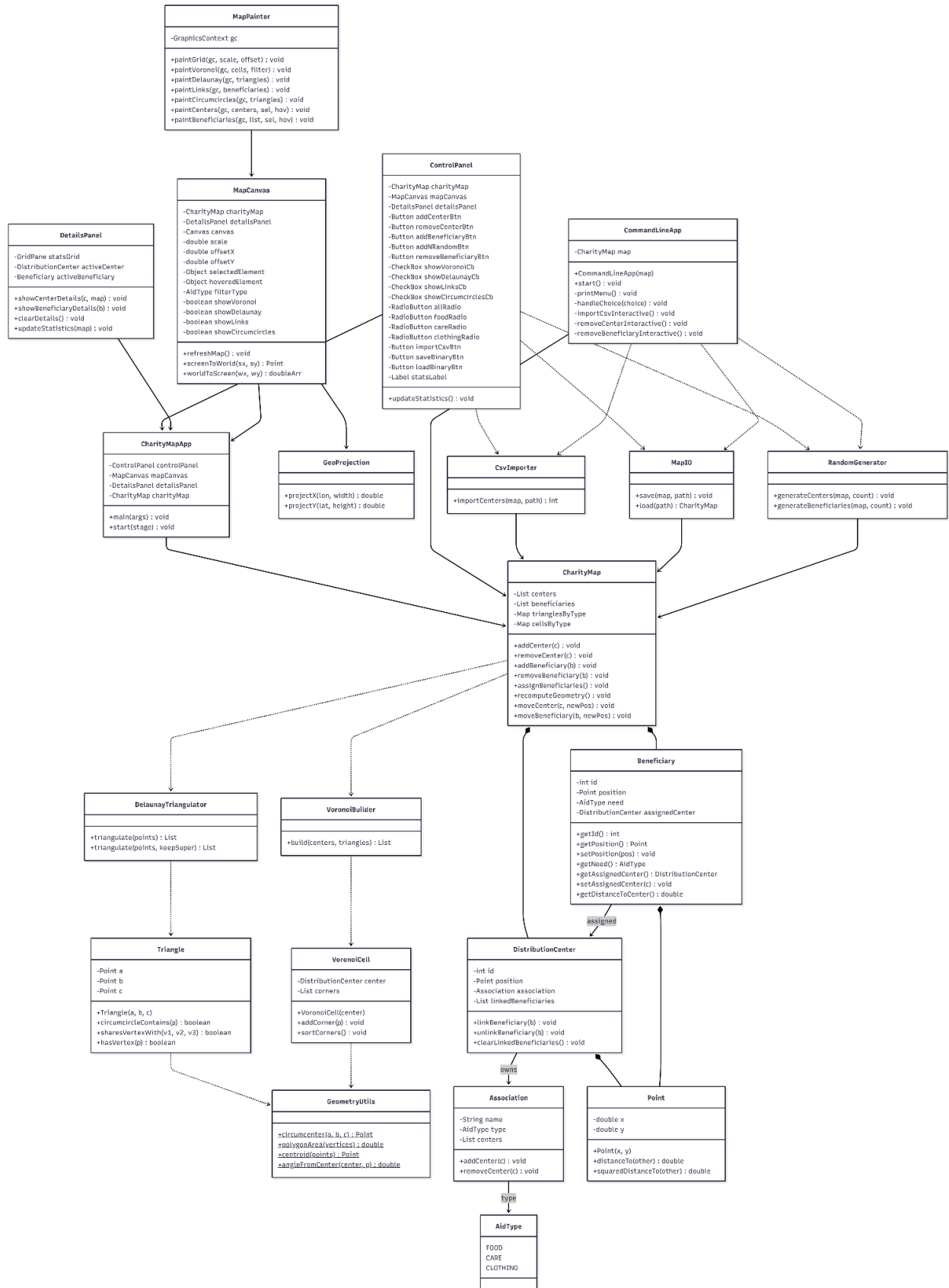


Figure 1 : Diagramme de classes UML complet et partitionné de l'application CharityMap

5. Choix techniques

Pour répondre aux contraintes du cahier des charges académique, nous avons privilégié des technologies standards mais puissantes de l'environnement Java :

- **Java 17 (LTS)** : Offre un compromis parfait de stabilité, de performances de calcul et d'accès aux structures modernes.
- **JavaFX** : Permet la gestion de composants visuels modernes (Canvas pour le dessin vectoriel, BorderPane pour la structuration globale, formulaires complexes).
- **Maven** : Automatise la compilation multiplateforme, le téléchargement des dépendances JavaFX et la génération automatisée de la Javadoc.
- **Sauvegarde automatisée (Serializable)** : Utilisée par la classe MapIO pour sauvegarder l'état mémoire d'une carte dans un format de fichier compact .map, préservant les liens logiques exacts.
- **Formule du lacet de soulier** : Implémentée pour le calcul précis de l'aire des cellules de Voronoï de forme polygonale quelconque à partir de leurs sommets triés par angle polaire.

6. Problèmes rencontrés et solutions

Durant le cycle de développement, plusieurs points de blocage algorithmiques et ergonomiques ont été résolus avec des choix d'ingénierie adaptés :

Problème rencontré	Cause racine	Solution retenue
Décalage géométrique après déplacement d'un point	Modifications des coordonnées sans relancer la triangulation.	Centralisation du recalcul automatique de Delaunay et Voronoï dans le contrôleur global après chaque action CRUD.
Affichage incohérent des cellules de Voronoï	Les sommets des polygones étaient reliés dans un ordre aléatoire, créant des polygones auto-entrecroisés.	Implémentation d'un tri polaire trigonométrique par angle par rapport au centre de secours dans VoronoiCell.sortCorners().
Lignes CSV invalides	Lignes vides, commentaires	Création d'un parseur

bloquant l'importation

ou coordonnées GPS
malformées.

**tolérant avec blocs try-
catch isolés. Les lignes
erronées sont ignorées au
lieu de faire planter
l'application.**

**Difficulté de test sans
l'interface graphique**

Dépendance forte des
fenêtres JavaFX au
démarrage.

**Isolation complète de la
logique métier et de calcul
dans les packages
model/geometry.
Utilisation de
CommandLineApp pour
valider les calculs.**

7. Tests et résultats

Pour valider le comportement algorithmique, une série de tests fonctionnels et de limites a été conduite :

- **Vérification du maillage** : Ajout de 3 à 50 points et contrôle de l'absence de chevauchement de triangles (propriété de Delaunay).
- **Simulation de charge** : Génération automatique de 100 bénéficiaires et contrôle de l'affectation immédiate au plus proche voisin compatible.
- **Robustesse d'importation** : Importation réussie d'un jeu de données de test CSV avec lignes vides et commentaires.
- **Sauvegarde et Restauration** : Sauvegarde binaire de l'état, fermeture de l'application, réouverture, rechargement du fichier et vérification de la concordance parfaite des positions géométriques.

Résultat : Les temps de calcul géométriques de Delaunay et Voronoï restent invisibles pour l'utilisateur (inférieurs à 5ms pour 100 entités). L'interface graphique est fluide lors du déplacement des points, et le modèle binaire de persistance est fonctionnel.

8. Cas d'utilisation

L'utilisateur principal de l'application est un coordinateur humanitaire ou un planificateur social qui souhaite visualiser une carte d'aide caritative. Il interagit avec l'interface pour insérer de nouveaux bénéficiaires, modifier la structure territoriale en déplaçant les centres, observer les statistiques géographiques (aires, distances) et persister son travail sur disque.

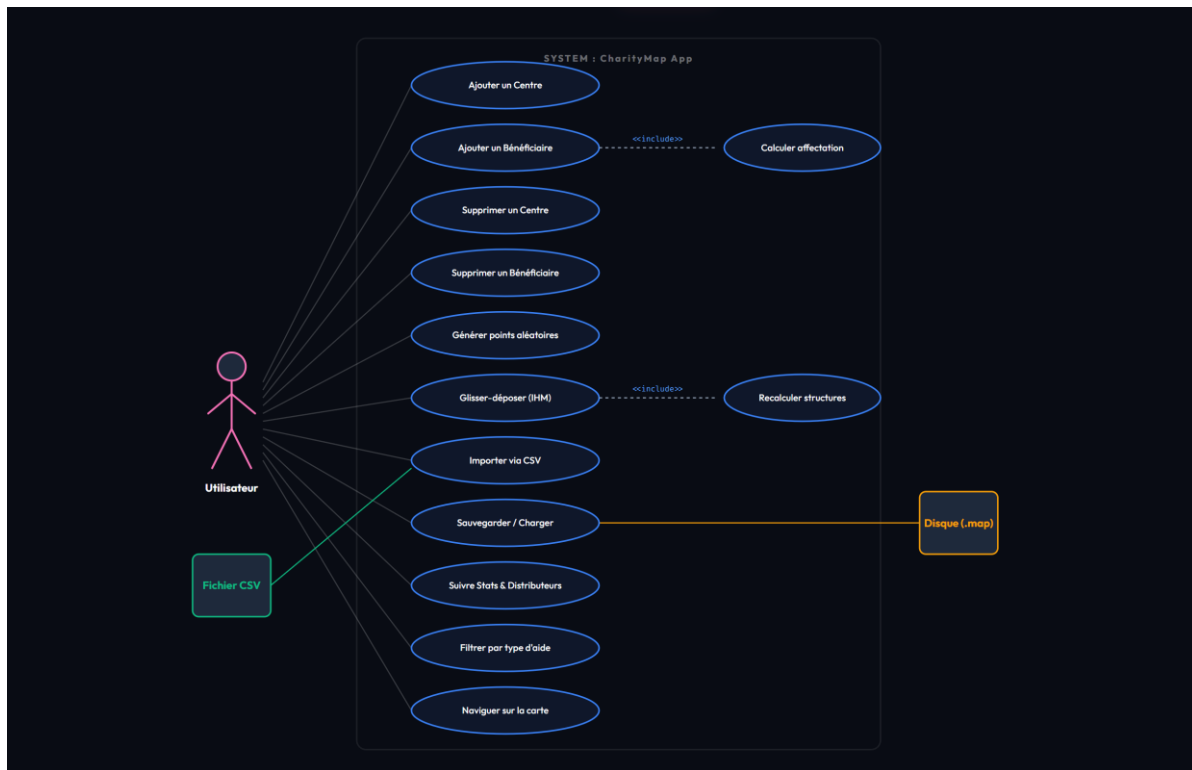


Figure 2 : Diagramme des cas d'utilisation (Use Case) de l'application CharityMap

9. Limites et améliorations possibles

- Intégrer des alertes d'interface bloquantes en cas de coordonnées aberrantes ou de noms de centres identiques ;
- Pour les points situés aux limites géographiques de la carte, limiter les cellules infinies de Voronoï par une boîte englobante fixe (Bounding Box) pour éviter les débordements visuels ;
- Ajouter des graphiques de répartition globale dans le DetailsPanel ;
- Rédiger des scripts de tests unitaires (JUnit) couvrant spécifiquement les calculs de Delaunay et de calcul d'aires dans GeometryUtils.

10. Conclusion

Le projet CharityMap a été une opportunité complète d'intégrer des notions de géométrie algorithmique complexes (Delaunay et Voronoï) au sein d'une architecture applicative structurée et découplée.

L'utilisation d'une problématique d'aide humanitaire a donné une dimension concrète à ces concepts mathématiques abstraits. L'application ne se limite pas à afficher des coordonnées, elle cartographie des besoins réels de couverture sociale. L'organisation du travail en branches

Git et par packages nous a sensibilisés aux exigences professionnelles du génie logiciel. La base applicative développée est saine, documentée et extensible.